1. ABSTRACT AND PROJECT OBJECTIVES

1.1 Abstract

Campus Route Finder is an AI application that models campus locations as a graph and demonstrates search and reasoning through a Streamlit dashboard. Breadth-First Search (BFS) finds a minimum-edge route in an unweighted graph. The Python extension compares BFS with Uniform Cost Search (UCS) and A* for cost-aware navigation, uses backtracking and AC-3 constraint propagation for timetable generation, and applies rule-based knowledge representation to identify suitable nearby facilities. The original Java BFS implementation is retained. The latest reported Python test run passed all 44 tests. Congestion values are illustrative scenario costs, not live traffic measurements.

1.2 Keywords

1. Artificial Intelligence
2. BFS
3. Uniform Cost Search
4. A* Search
5. Constraint Satisfaction
6. AC-3
7. Streamlit.

1.3 Problem statement:- Campus Route Finder using BFS · *BFS*

- **Build:** Find the shortest route between two campus locations on an unweighted graph using breadth-first search.
- **Requirements:** A defined graph; BFS implementation; printed/visualised path and hop count.

Must demonstrate: Why BFS gives the fewest-edge path on an unweighted graph.

1.4 Objectives

1. Implement BFS for minimum-edge routing.
2. Compare BFS with UCS and A* when edge costs differ.
3. Generate and validate timetable assignments using backtracking and AC-3.
4. Demonstrate rule-based selection of nearby facilities.
5. Present outputs through Streamlit and test the implemented modules.

1.5 Scope

Included manually defined campus graph, route finding, cost-aware scenarios, timetable CSP, AC-3, facility knowledge rules, Streamlit UI, retained Java code, and Python tests. Excluded: live GPS, verified live traffic, production timetable integration, and comprehensive real-world accessibility certification.

2. INTRODUCTION AND PROPOSED SOLUTION

2.1 Background

Campus navigation and scheduling are practical settings for introductory AI. Graph search finds routes through connected locations; constraint satisfaction searches for assignments that satisfy rules; knowledge representation stores facts about facilities. A dashboard makes the outputs easier to inspect.

2.2 Existing system / current approach

Users may rely on memory, signage, static maps, or manually listed routes. Timetables may be prepared in spreadsheets and checked manually. Facility information may be stored separately, requiring users to ask staff or inspect room details.

2.3 Limitations of the existing approach

Manual route choices may not minimise hops or account for differing edge costs. Manual timetable preparation can miss conflicts when room, faculty, batch, and travel constraints interact. Facility facts stored separately make nearby-suitable-facility queries harder to answer consistently.

2.4 Proposed AI-based solution

The campus is represented as a graph. BFS explores level by level to minimise hops; UCS and A* use configured non-negative edge costs to find minimum-cost paths. Backtracking searches timetable assignments and checks constraints, while AC-3 prunes incompatible domain values. A rule-based module filters facilities using encoded properties and graph distance. Streamlit presents results.

2.5 Expected benefits / intended users

Potential users include students, campus visitors, teaching staff, and AI project evaluators. Intended benefits are transparent route explanations, comparison of search objectives, a timetable-constraint demonstration, and an inspectable facility-selection prototype.

3. AI CONCEPTS AND PROBLEM FORMULATION

3.1 AI concept used

The project combines state-space search (BFS, UCS, A*), constraint satisfaction (backtracking and AC-3), and rule-based knowledge representation.

3.2 Why this AI concept is suitable for the problem

BFS is appropriate when every edge represents one hop with equal cost. UCS is appropriate for non-negative edge costs. A* uses a heuristic to guide exploration, with optimality depending on the heuristic and cost assumptions. Timetable generation is a CSP because variables must be assigned values while satisfying rules. Explicit facility facts suit a small rule-based knowledge base.

3.3 Problem formulation

Route search: initial state is the selected start; actions move to adjacent locations; goal is the destination; BFS cost is number of edges, while UCS/A* minimise configured accumulated cost. Timetable CSP: variables are scheduled classes, domains are candidate time-room assignments, and constraints enforce room capacity/type, room/faculty/batch conflict rules, and configured distance conditions. Facility reasoning takes a starting location and desired properties, filters matching facilities, and prefers nearby candidates.

3.4 PEAS description

Performance measure: valid route or timetable, minimum hops/cost under the selected objective, constraint satisfaction, and understandable output. Environment: campus graph, timetable data, and facility knowledge base. Actuators: displayed routes, hop counts, costs, schedules, and recommendations. Sensors/inputs: user selections, graph data, scenario settings, timetable domains/constraints, and facility criteria.

Component	Project-specific description
Performance measure	Valid output; minimum hops/cost; constraint satisfaction; clarity.
Environment	Campus graph, timetable data, facility facts.
Actuators	Routes, costs, schedules, recommendations displayed.
Sensors / inputs	User selections, graph, scenario settings, constraints and criteria.

3.5 Input, output and assumptions

Inputs include route endpoints, graph connections, blocked-edge and estimated-congestion settings, timetable domains and constraints, and desired facility properties. Outputs include paths, hop counts, estimated costs, timetable assignments, validation results, and candidate facilities. Graph and facts are manually configured. No live traffic or GPS feed is claimed.

4. ALGORITHM AND METHODOLOGY

4.1 Methodology

The project began with a campus graph and BFS. A Streamlit interface was added while retaining Java source. Further modules implemented cost-aware navigation, timetable CSP, AC-3 propagation, and facility reasoning. Automated tests cover route cases, cost comparisons, timetable validity, constraints, and integration.

4.2 Algorithm / pseudocode

- BFS: enqueue the start and mark it visited; repeatedly dequeue a location; if it is the goal, reconstruct the route from parent links; otherwise enqueue unvisited neighbours and record parents; if the queue empties, report no route.

- UCS expands the frontier item with the lowest accumulated cost.

- A* prioritises accumulated cost plus a heuristic. Backtracking assigns one timetable variable at a time, checks consistency, recurses, and undoes failed choices.

- AC-3 processes constraint arcs and removes values with no compatible neighbour-domain value.

4.3 Explanation of the AI technique

BFS finds a minimum-edge path because nodes are visited in nondecreasing depth. UCS minimises accumulated non-negative cost. A* combines accumulated cost with a heuristic; optimality depends on the heuristic and cost model. Backtracking explores assignments and rejects partial assignments that violate constraints. AC-3 reduces domains and may reduce search. Facility reasoning filters candidates by stored properties and graph distance.

4.4 Data structures / knowledge representation used

BFS uses an adjacency graph, queue, visited set, and parent map. Cost-aware search maintains a priority frontier and best-known costs. CSP uses variables, domains, and constraints; AC-3 processes constraint arcs. Facility reasoning uses structured facts and rules.

4.5 Time and space complexity

For an adjacency-list graph with V vertices and E edges, BFS takes O(V + E) time and O(V) auxiliary space. UCS/A* performance depends on graph and cost structure. AC-3 is commonly bounded by O(Ec d^3), where Ec is the number of constraint arcs and d is maximum domain size. Backtracking CSP search is exponential in the worst case.

5. IMPLEMENTATION AND GITHUB REPOSITORY

5.1 Development environment

1. Developed on macOS using Terminal.
2. Reported environment: Python 3.14.0,
3. Streamlit 1.65.0,
4. pytest 9.1.1,
5. Java source files retained in the repository.

5.2 Implementation steps

1. Define campus locations and edges.
2. Implement and test BFS.
3. Add Streamlit.
4. Add UCS/A* and scenario costs.
5. Implement timetable constraints and backtracking.
6. Add AC-3.
7. Add facility facts and rules.
8. Run tests and update documentation.

5.3 GitHub repository URL:- https://github.com/darshini6-ai/campus-route-finder

5.4Repository structure

```
.
./.streamlit
./.streamlit/config.toml
./app.py
./dependencies.txt
./docs
./docs/Campus Route Finder (IA-2).pdf
./docs/report.pdf
./LICENSE
./pytest.ini
./python_src
./python_src/__init__.py
./python_src/bfs.py
./python_src/campus_graph.py
./python_src/campus_knowledge.py
./python_src/csp
./python_src/csp/__init__.py
./python_src/csp/timetable_csp.py
./python_src/csp/timetable_data.py
./python_src/smart_navigation.py
./README.md
./requirements.txt
./src
./src/__init__.py
./src/BFS.java
./src/CampusGraph.java
./src/Location.java
./src/Main.java
./src/smartcampus
./src/smartcampus/__init__.py
./src/smartcampus/csp
./src/smartcampus/graph
./tests
./tests/BFSTest.java
./tests/test_ac3_backtracking.py
./tests/test_ac3.py
./tests/test_astar.py
./tests/test_backtracking.py
./tests/test_bfs.py
./tests/test_constraints.py
./tests/test_smart_navigation.py
./tests/test_timetable_csp.py
./tests/test_timetable_integration.py
./tests/test_ucs.py
```

5.5 Setup and execution instructions

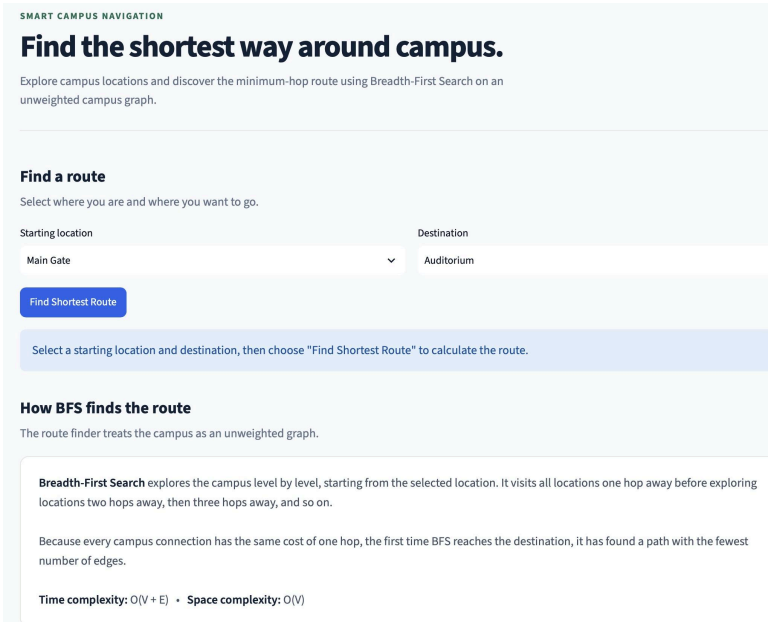
From the repository root: `python3 -m pip install -r requirements.txt`; launch with `python3 -m streamlit run app.py`; test with `python3 -m pytest -v`.

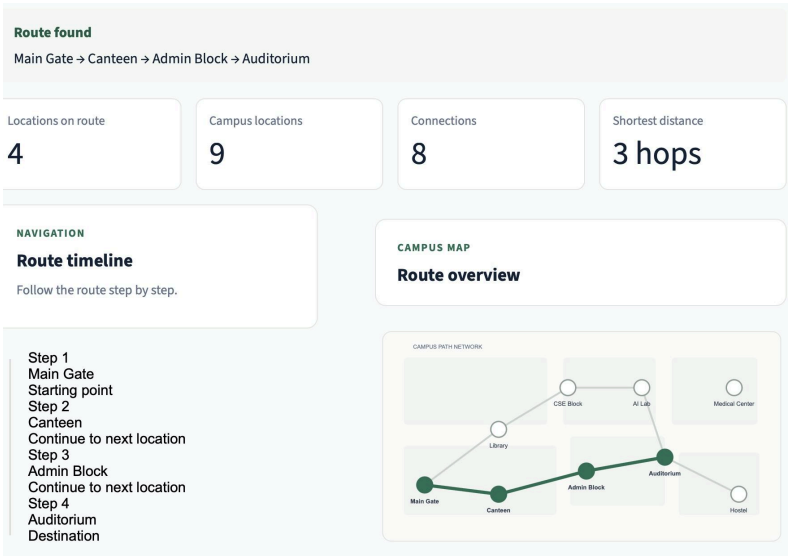
The last reported run collected 44 tests and all 44 passed in 0.03 seconds. With JDK installed, the original Java app can be compiled with `javac -d out src/*.java` and run with `java -cp out Main`.

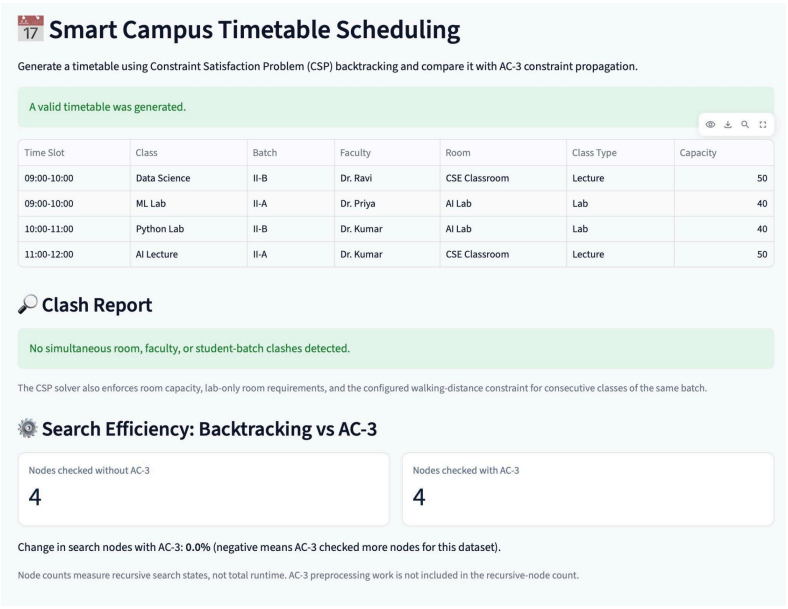
5.6 Commit history / contribution summary

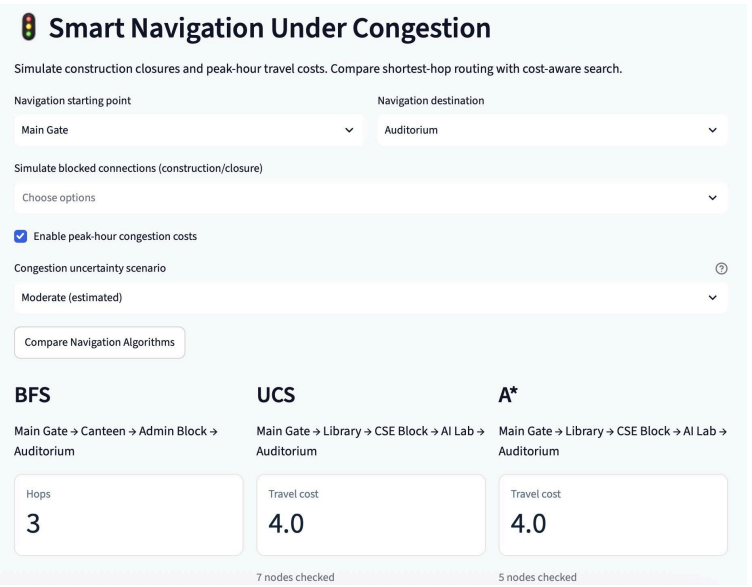
The author committed the dashboard and AI modules, added the updated IA-2 PDF, updated the README, and pushed commits to GitHub. The PDF was committed in 93206cd and the README update in 364ee1f. The annotated v2.0 tag was created and pushed. Add team contributions only if applicable.

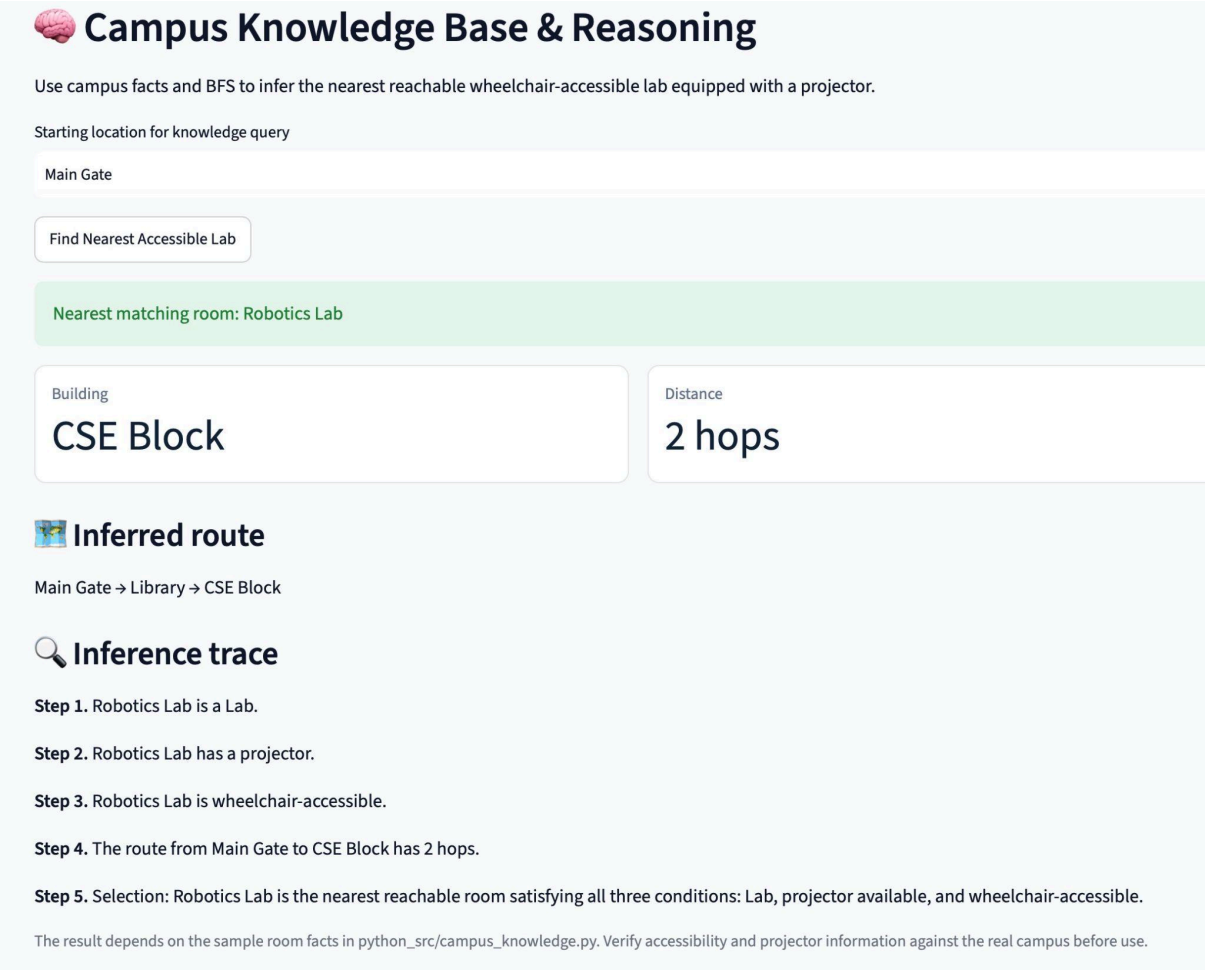
6. RESULTS AND OBSERVATIONS











6.1 Results summary

The latest reported automated test run collected 44 tests and reported 44 passed in 0.03 seconds. Tests cover BFS, UCS, A*, timetable constraints, AC-3, backtracking, and integration. The dashboard demonstrates minimum-hop and cost-aware routes, timetable assignments, and facility reasoning. Genuine screenshots from the final running dashboard should be inserted where indicated.

6.2 Observations

Route tests include normal paths, same start and destination, and unreachable destinations. A smart-navigation test checks that BFS optimises hops while UCS/A* optimise configured cost. CSP tests check room suitability, capacity, faculty/batch conflicts, distance constraints, and timetable validity. These results support the tested cases, not every possible campus scenario.

6.3 Performance measure / accuracy / success rate

Automated test pass rate in the last reported run: 44/44 = 100%. This is a test pass rate, not a model accuracy metric. A previously captured small timetable example showed four recursive nodes for each method, with AC-3 preprocessing excluded; do not generalise this small example. Congestion costs are estimates, not measured travel-time predictions.

7. DISCUSSION, LIMITATIONS AND FUTURE ENHANCEMENTS

7.1 Discussion

The appropriate search algorithm depends on the objective. BFS minimises hops in an unweighted graph; UCS/A* minimise configured costs. CSP methods search for consistent timetable assignments. Rule-based knowledge makes facility properties explicit and inspectable.

7.2 Limitations

1. Graph and facility facts are manually encoded.
2. Congestion values are illustrative, not live traffic.
3. Accessibility claims require real-world verification.
4. A* depends on heuristic quality.
5. Timetable results are limited by the variables, domains, and constraints encoded.
6. Passing tests does not establish production readiness.

7.3 Future enhancements

1. Use verified campus maps, walking distances, and accessible routes.
2. Integrate reliable traffic data and validate edge costs.
3. Benchmark backtracking with and without AC-3 on larger timetables.
4. Add explicit building/floor relationships and fact provenance.
5. Include reproducible benchmark outputs and genuine screenshots**.**

7.4 Learning outcomes

The project provides practice in graph modelling, BFS, cost-aware and heuristic search, CSP formulation, backtracking, arc consistency, knowledge representation, testing, Streamlit, Git branching, commits, tags, and technical documentation.

8. CONCLUSION AND REFERENCES

In conclusion, the Smart Campus Navigation and Timetabling System successfully demonstrates how Artificial Intelligence techniques can be applied to solve practical campus management and navigation problems. Breadth-First Search (BFS) identifies the shortest path in terms of the number of connections, while Uniform Cost Search (UCS) and A* Search helps find cost-effective routes under simulated congestion and road closures. The timetable scheduling module uses Constraint Satisfaction Problem (CSP) techniques, backtracking, and AC-3 constraint propagation to generate valid timetables while considering room capacity, laboratory requirements, faculty and batch conflicts, and walking-distance constraints. The knowledge-based reasoning module demonstrates how room properties such as accessibility and projector availability can help identify suitable campus facilities. The Streamlit dashboard provides an interactive interface for exploring these features and comparing algorithm results. Overall, this project strengthens the understanding of search algorithms, constraint satisfaction, knowledge representation, and responsible AI while providing a foundation for future improvements, such as real-time congestion data, verified accessibility information, and more advanced campus scheduling.